COURSE CODE: DFR101

LAB TITLE: CIP-B101_LAB1A_Number_Sytems_ASCII_Timestamps_Data_Representations

FULL NAME: GRACE MCHIBUMA JIBANIYA

STUDENT ID: c11.pcdfir2617045

DATE: 09 – 11 – 2026

PLATFORM USED: Windows and Kali VM (Virtual Box)

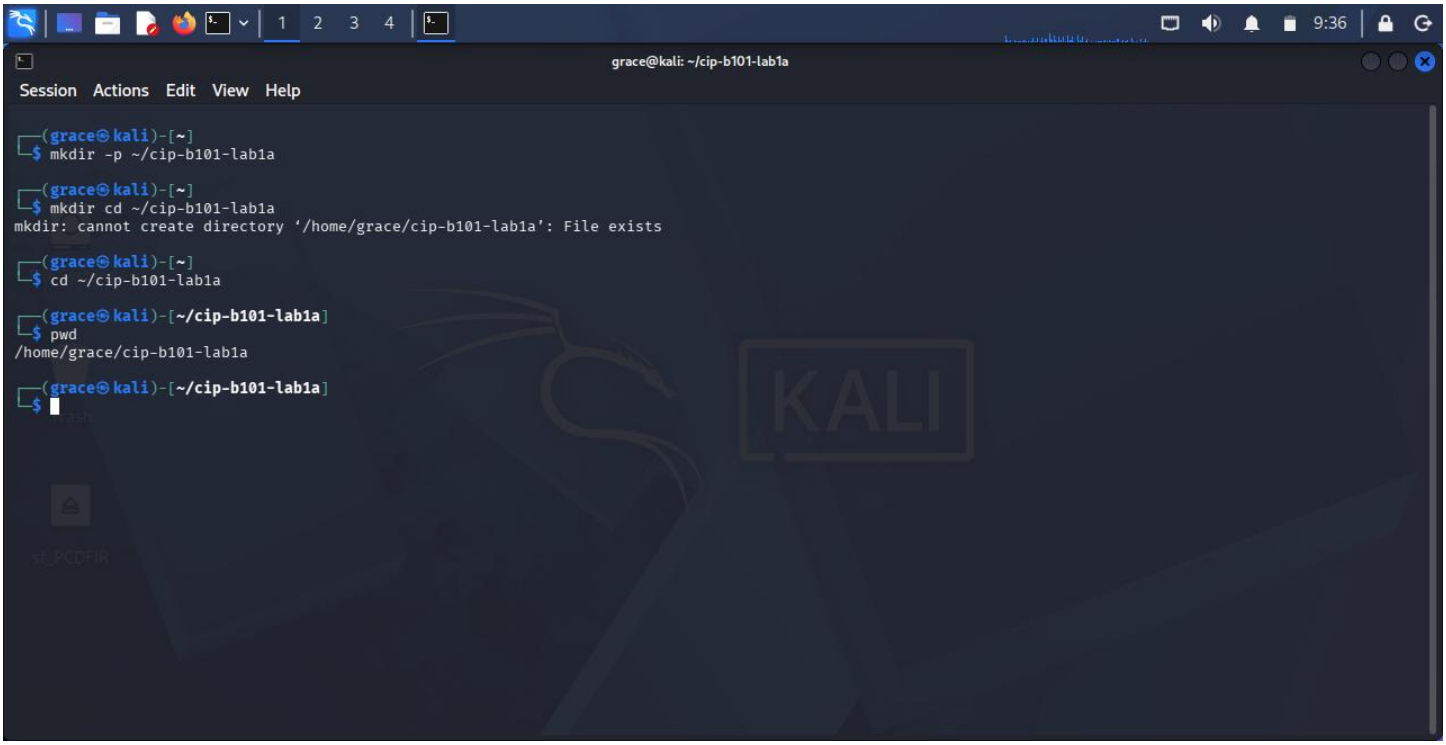
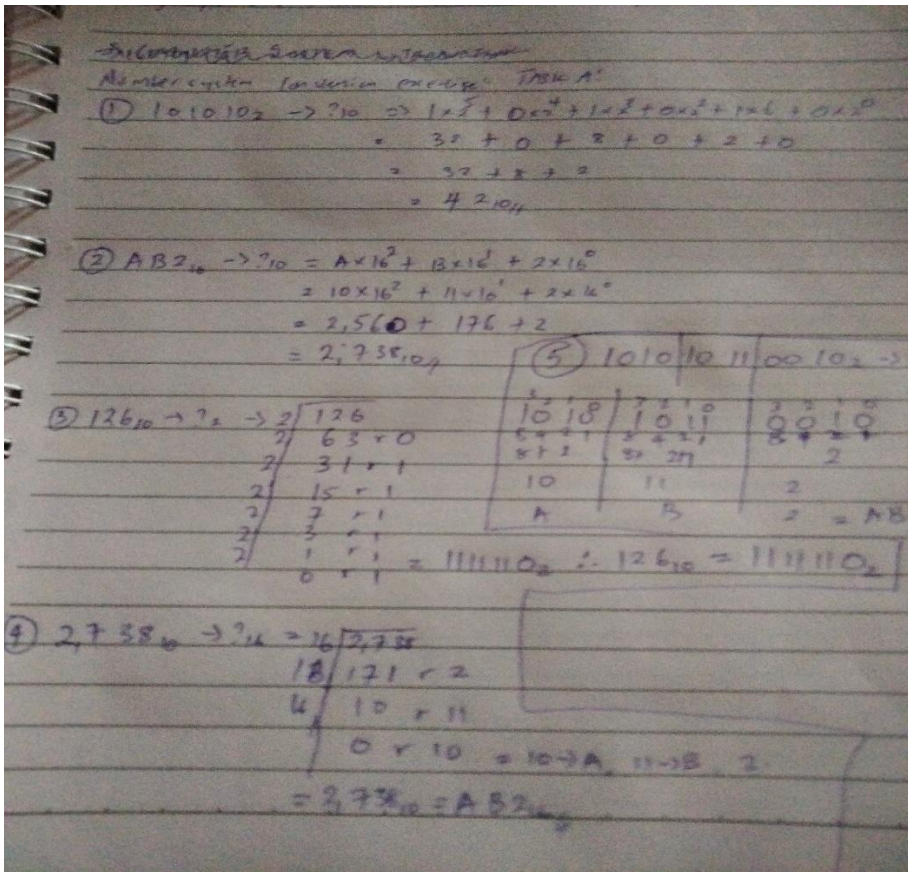


Fig.1

Task A - Number Systems and Manual Conversion

Evidence Value	Given Base	Manual Answer	Command Used to Verify	Screenshot Ref.
101010	Binary	42		Fig.2
AB2	Hexadecimal	2738		Fig.2
126	Decimal	1111110		Fig.2
2738	Decimal	AB2		Fig.3
101010110010	Binary	AB2		Fig.3



```
grace@kali: ~/cip-b101-lab1a
Session  Actions  Edit  View  Help

┌──(grace㉿kali)-[~/cip-b101-lab1a]
└─$ echo "$((2#101010))"
42

┌──(grace㉿kali)-[~/cip-b101-lab1a]
└─$ # hexadecimal to decimal using shell arithmetic

┌──(grace㉿kali)-[~/cip-b101-lab1a]
└─$ echo "$((16#AB2))"
2738

┌──(grace㉿kali)-[~/cip-b101-lab1a]
└─$ sudo apt install bc
[sudo] password for grace:
bc is already the newest version (1.07.1-4).
bc set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0

┌──(grace㉿kali)-[~/cip-b101-lab1a]
└─$ echo "obase=2;126" | bc
1111110

┌──(grace㉿kali)-[~/cip-b101-lab1a]
└─$ echo "obase=16;2738" | bc
AB2

┌──(grace㉿kali)-[~/cip-b101-lab1a]
└─$ printf "%x" "2738"
ab2
┌──(grace㉿kali)-[~/cip-b101-lab1a]
└─$
```

Fig.2

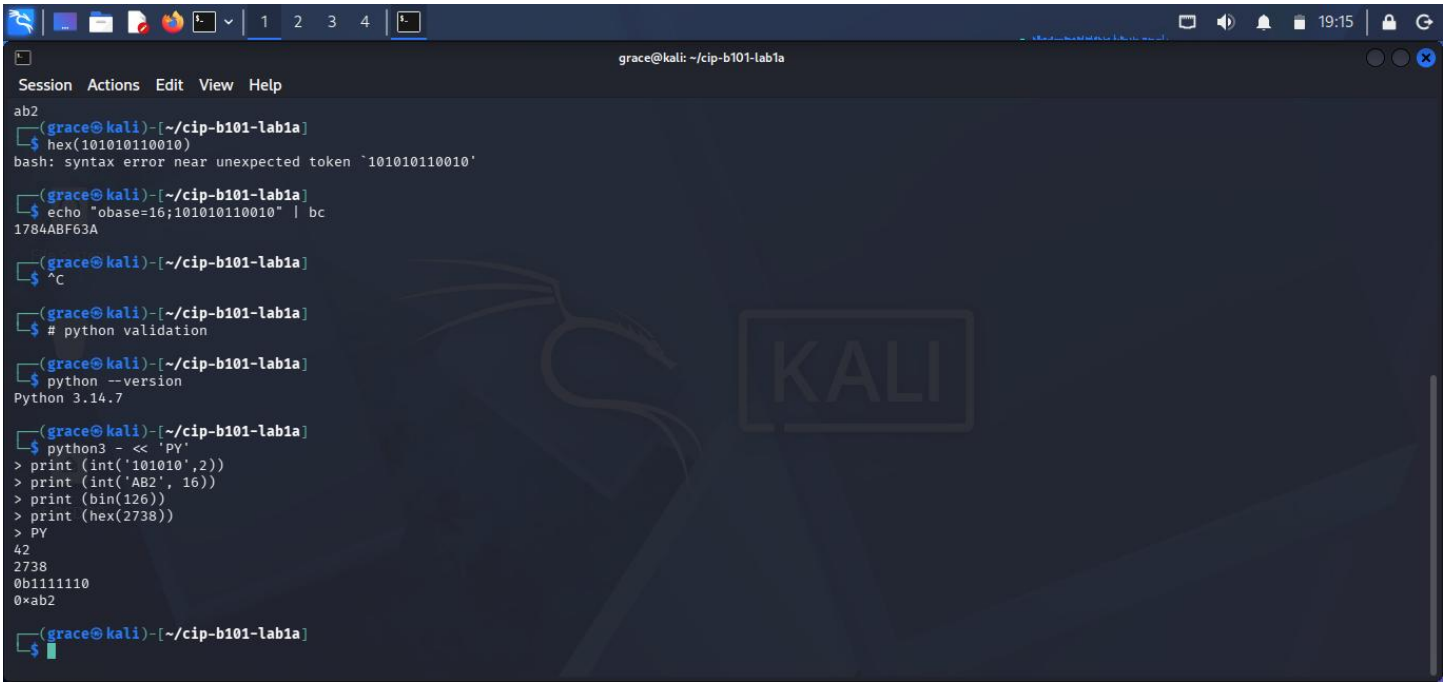


Fig.3

Task B - ASCII, UTF-8 and Hex Evidence Strings

1. Explain why 0A may appear at the end of output when echo is used without -n.: 0A is the hexadecimal representation of a newline character (line feed). By default, the echo command adds a newline after the text it prints. Therefore, when the output is examined in hexadecimal form, an additional 0A may appear at the end. Using echo -n prevents echo from adding this trailing newline
2. explain why an examiner should not assume every hexadecimal value is automatically readable text: An examiner should not assume that every hexadecimal value represents readable text because hexadecimal is simply a way of representing binary data. The bytes may represent text encoded using ASCII or UTF-8, but they could also represent numbers, timestamps, file signatures, encrypted data, compressed data, executable instructions, or other binary information. The context and encoding must therefore be considered before interpreting hexadecimal data as text.

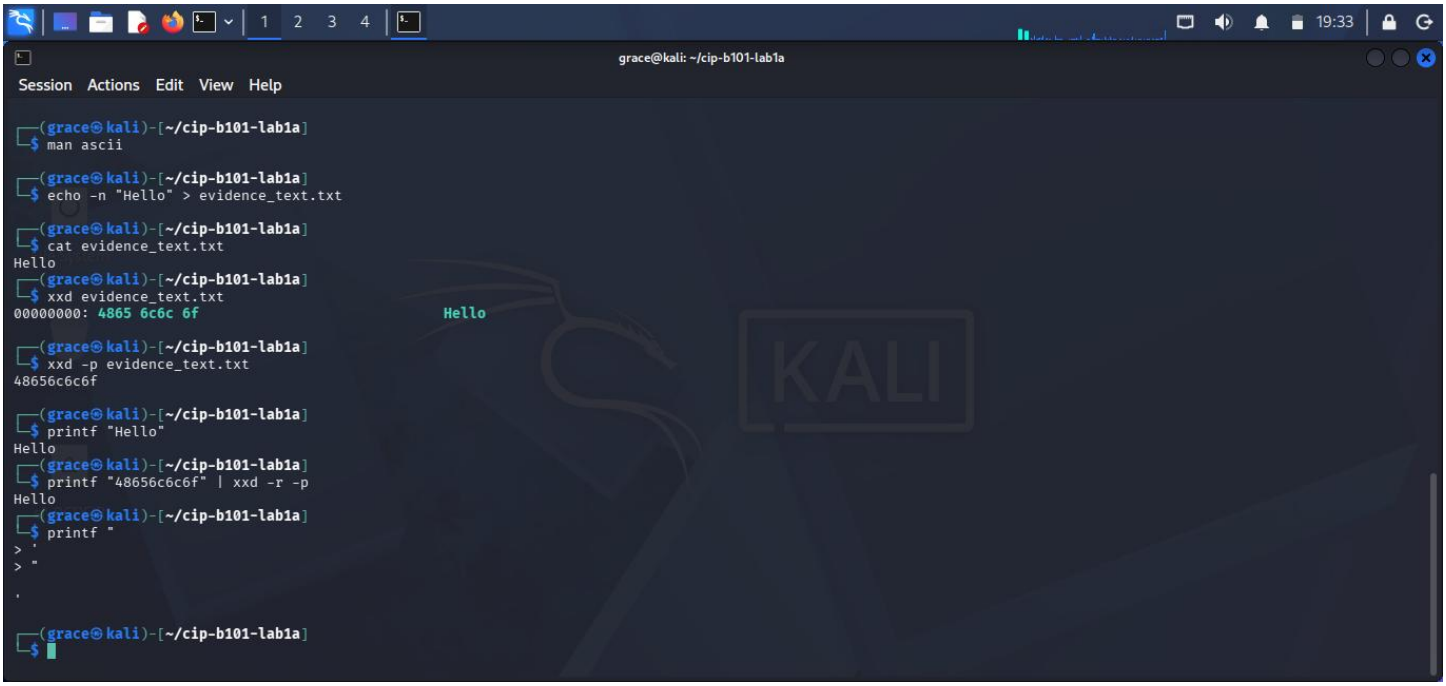

grace@kali: ~/cip-b101-lab1a
Session Actions Edit View Help
(grace㉿kali)-[~/cip-b101-lab1a]
$ man ascii
(grace㉿kali)-[~/cip-b101-lab1a]
$ echo -n "Hello" > evidence_text.txt
(grace㉿kali)-[~/cip-b101-lab1a]
$ cat evidence_text.txt
Hello
(grace㉿kali)-[~/cip-b101-lab1a]
$ xxd evidence_text.txt
00000000: 4865 6c6c 6f Hello
(grace㉿kali)-[~/cip-b101-lab1a]
$ xxd -p evidence_text.txt
48656c6c6f
(grace㉿kali)-[~/cip-b101-lab1a]
$ printf "Hello"
Hello
(grace㉿kali)-[~/cip-b101-lab1a]
$ printf "48656c6c6f" | xxd -r -p
Hello
(grace㉿kali)-[~/cip-b101-lab1a]
$ printf "
> '
> "
'
(grace㉿kali)-[~/cip-b101-lab1a]
$

Fig.4

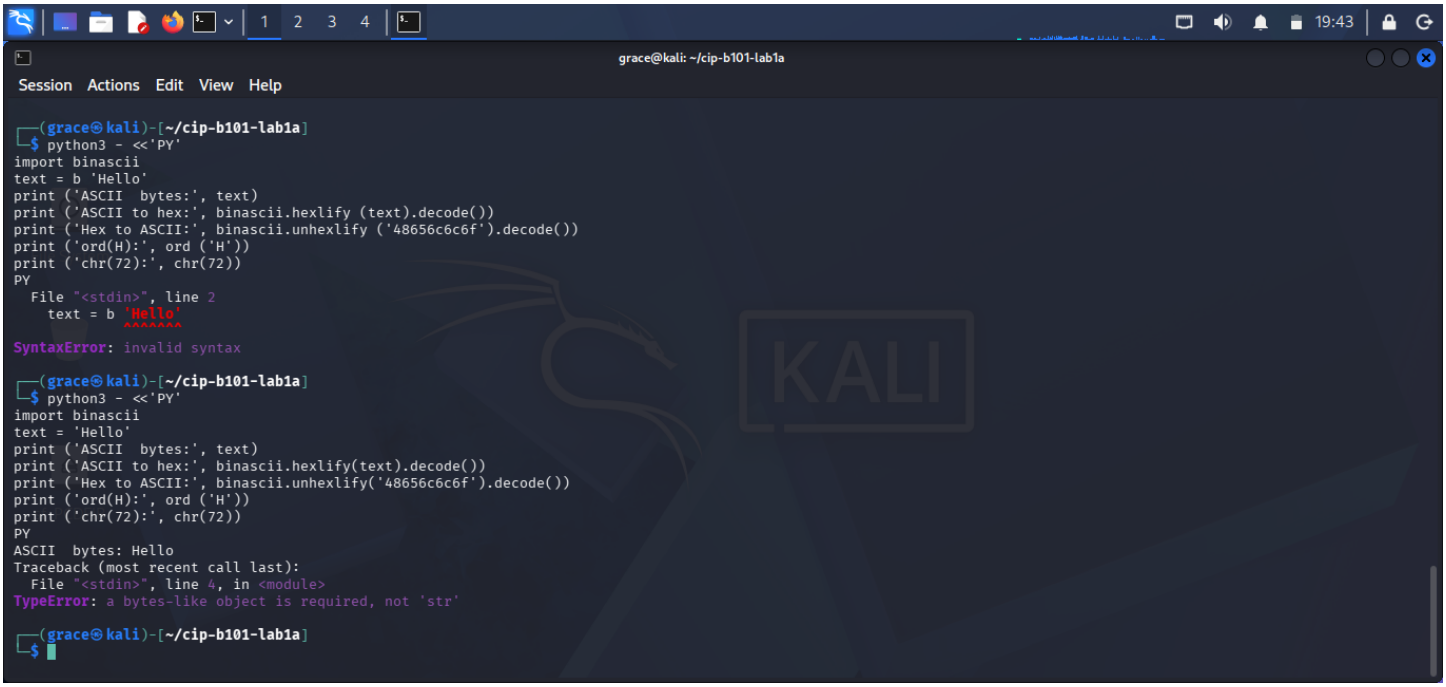

grace@kali: ~/cip-b101-lab1a
Session Actions Edit View Help
(grace㉿kali)-[~/cip-b101-lab1a]
$ python3 - <<'PY'
import binascii
text = b 'Hello'
print ('ASCII bytes:', text)
print ('ASCII to hex:', binascii.hexlify (text).decode())
print ('Hex to ASCII:', binascii.unhexlify ('48656c6c6f').decode())
print ('ord(H):', ord ('H'))
print ('chr(72):', chr(72))
PY
File "<stdin>", line 2
text = b 'Hello'
^^^^^^^
SyntaxError: invalid syntax
(grace㉿kali)-[~/cip-b101-lab1a]
$ python3 - <<'PY'
import binascii
text = 'Hello'
print ('ASCII bytes:', text)
print ('ASCII to hex:', binascii.hexlify(text).decode())
print ('Hex to ASCII:', binascii.unhexlify('48656c6c6f').decode())
print ('ord(H):', ord ('H'))
print ('chr(72):', chr(72))
PY
ASCII bytes: Hello
Traceback (most recent call last):
File "<stdin>", line 4, in <module>
TypeError: a bytes-like object is required, not 'str'
(grace㉿kali)-[~/cip-b101-lab1a]
$

Fig.5

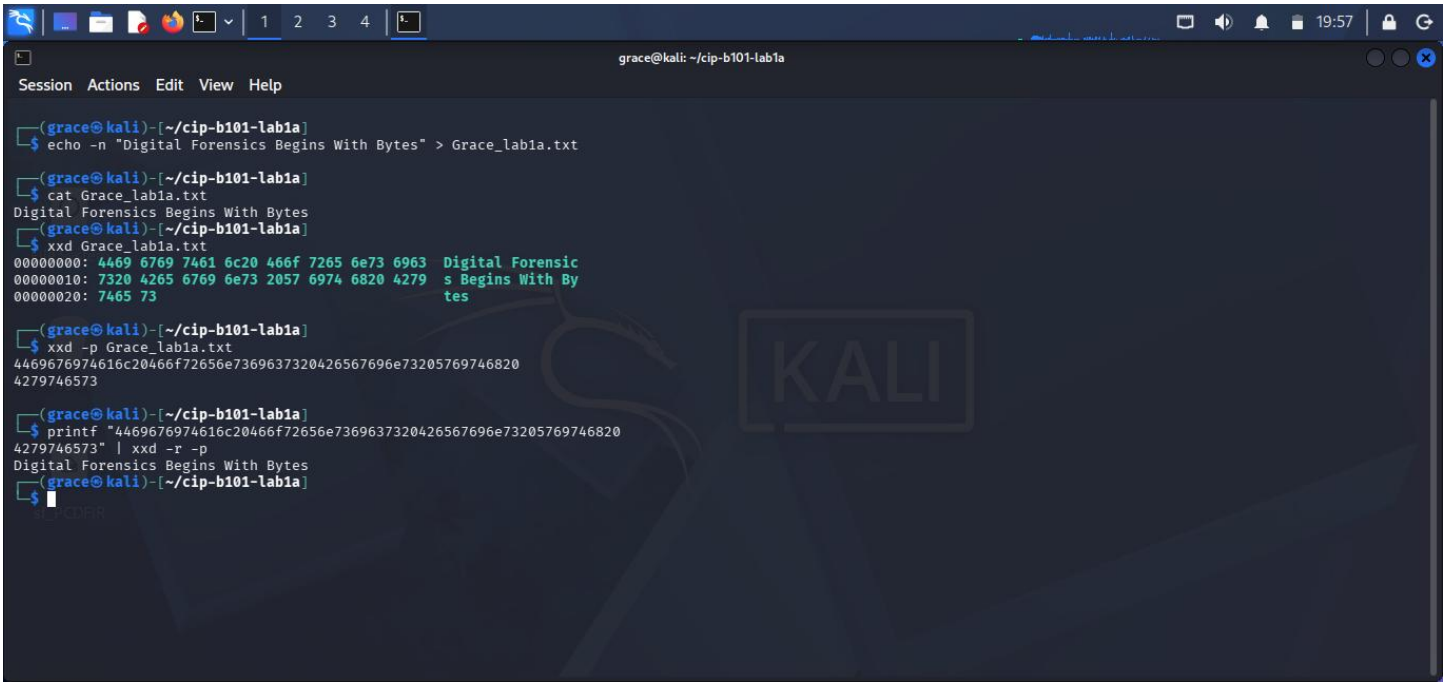

grace@kali: ~/cip-b101-lab1a
Session Actions Edit View Help
(grace㉿kali)-[~/cip-b101-lab1a]
$ echo -n "Digital Forensics Begins With Bytes" > Grace_lab1a.txt
(grace㉿kali)-[~/cip-b101-lab1a]
$ cat Grace_lab1a.txt
Digital Forensics Begins With Bytes
(grace㉿kali)-[~/cip-b101-lab1a]
$ xxd Grace_lab1a.txt
00000000: 4469 6769 7461 6c20 466f 7265 6e73 6963 Digital Forensic
00000010: 7320 4265 6769 6e73 2057 6974 6820 4279 s Begins With By
00000020: 7465 73 tes
(grace㉿kali)-[~/cip-b101-lab1a]
$ xxd -p Grace_lab1a.txt
4469676974616c20466f72656e736963732042656769 6e73205769746820
4279746573
(grace㉿kali)-[~/cip-b101-lab1a]
$ printf "4469676974616c20466f72656e736963732042656769 6e73205769746820
4279746573" | xxd -r -p
Digital Forensics Begins With Bytes
(grace㉿kali)-[~/cip-b101-lab1a]
$

Fig.6

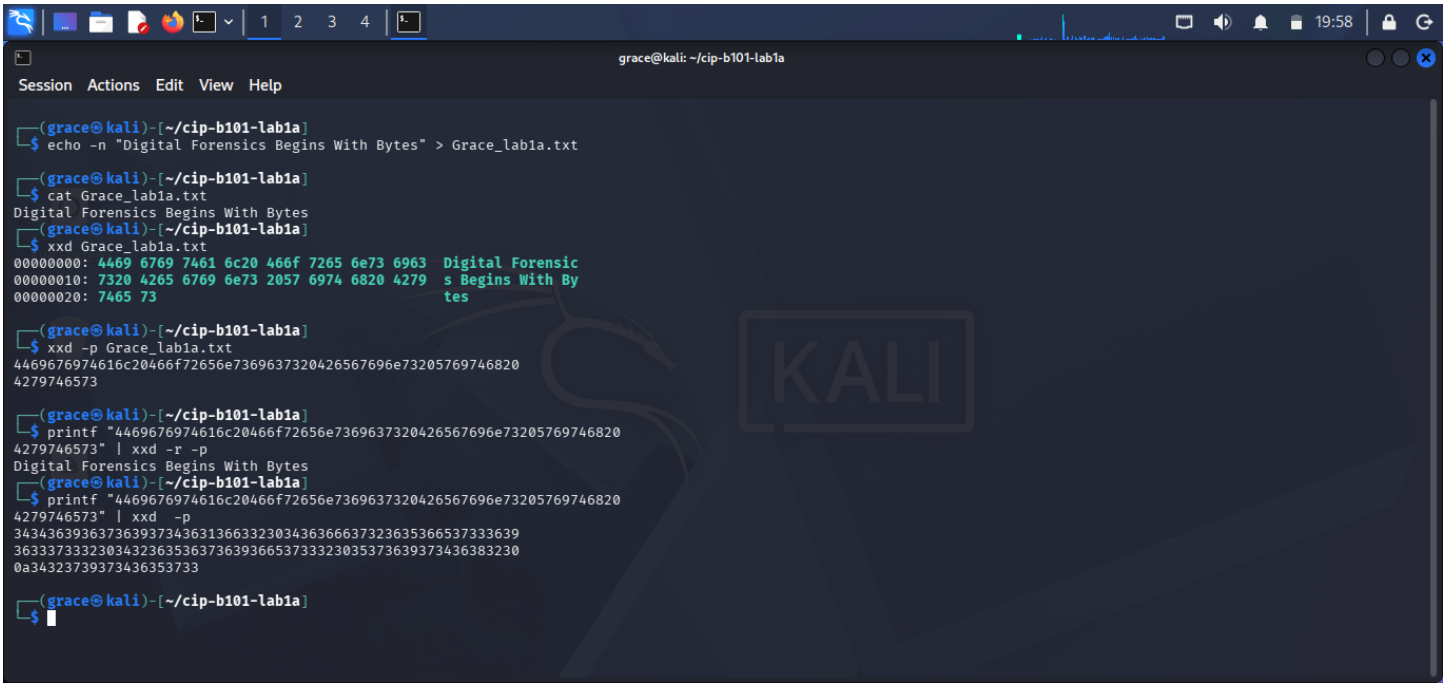

grace@kali: ~/cip-b101-lab1a
Session Actions Edit View Help
(grace㉿kali)-[~/cip-b101-lab1a]
$ echo -n "Digital Forensics Begins With Bytes" > Grace_lab1a.txt
(grace㉿kali)-[~/cip-b101-lab1a]
$ cat Grace_lab1a.txt
Digital Forensics Begins With Bytes
(grace㉿kali)-[~/cip-b101-lab1a]
$ xxd Grace_lab1a.txt
00000000: 4469 6769 7461 6c20 466f 7265 6e73 6963 Digital Forensic
00000010: 7320 4265 6769 6e73 2057 6974 6820 4279 s Begins With By
00000020: 7465 73 tes
(grace㉿kali)-[~/cip-b101-lab1a]
$ xxd -p Grace_lab1a.txt
4469676974616c20466f72656e736963732042656769 6e73205769746820
4279746573
(grace㉿kali)-[~/cip-b101-lab1a]
$ printf "4469676974616c20466f72656e736963732042656769 6e73205769746820
4279746573" | xxd -r -p
Digital Forensics Begins With Bytes
(grace㉿kali)-[~/cip-b101-lab1a]
$ printf "4469676974616c20466f72656e736963732042656769 6e73205769746820
4279746573" | xxd -p
3434363936373639373436313663323034363666373236353665373339
3633373332303432363536373639366537333230353736393734363832 30
0a34323739373436353733
(grace㉿kali)-[~/cip-b101-lab1a]
$

Fig.7

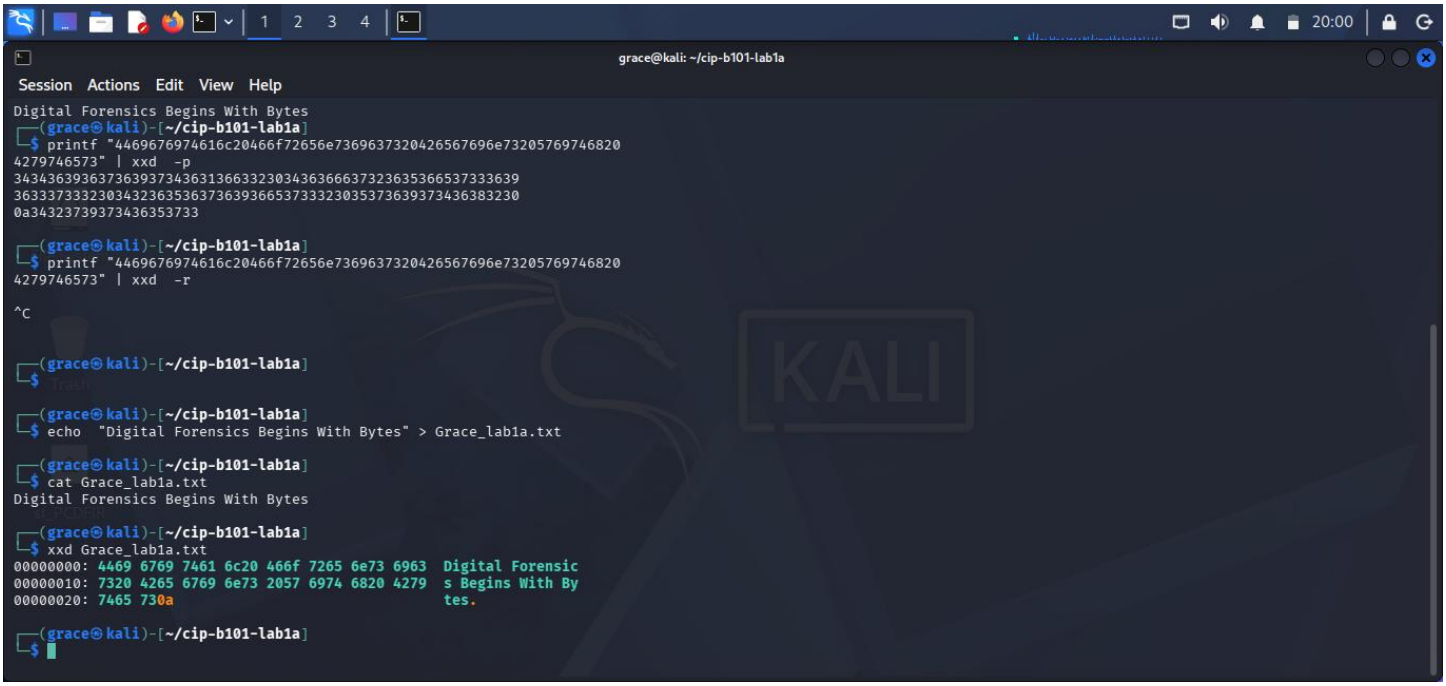


Fig.8

Task C - Epoch Time and Forensic Timestamps

Timestamp Value	Format	Conversion Command	Readable UTC Result	Screenshot Ref.
1643643491	Unix seconds			Fig.10
1675008832	Unix seconds			Fig.10
725846400	Mac Absolute Time seconds			Fig.11
Current time	Unix seconds	1794212301	Mon 09 Nov 2026 WAT	Fig.9

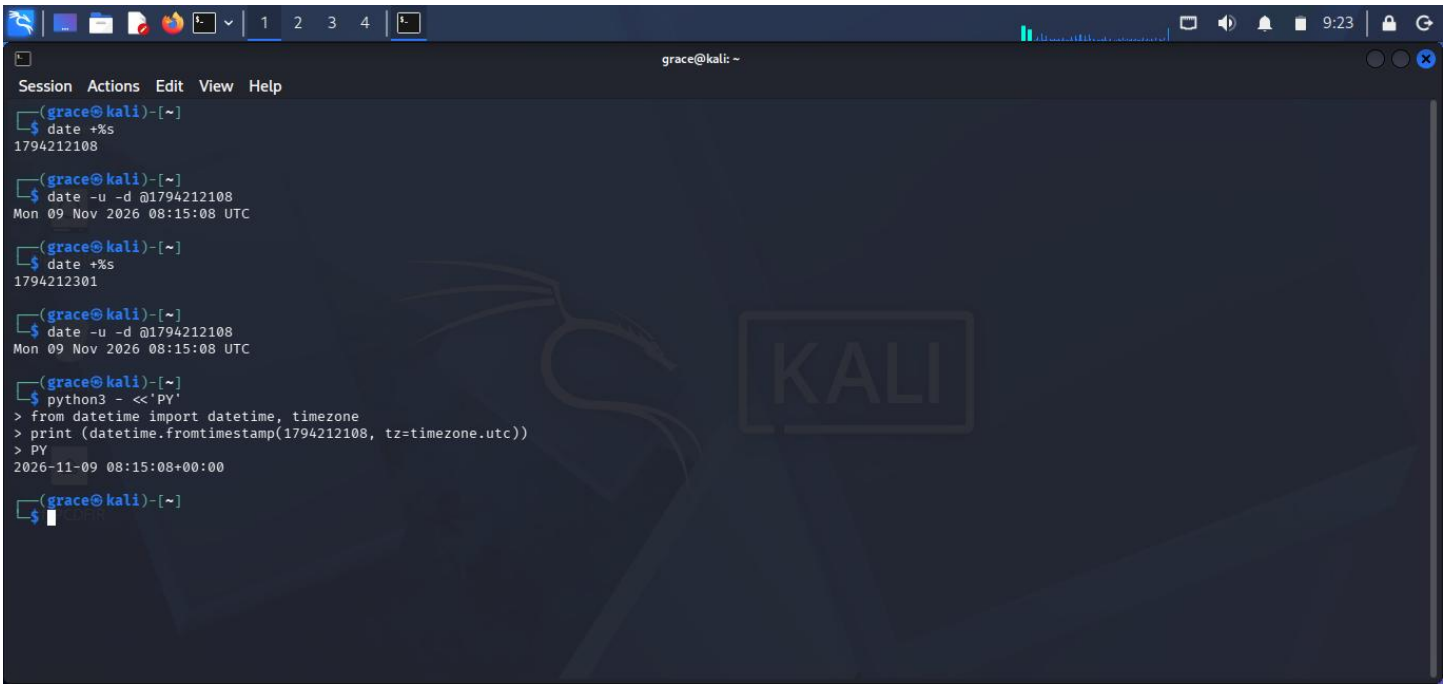


Fig.9

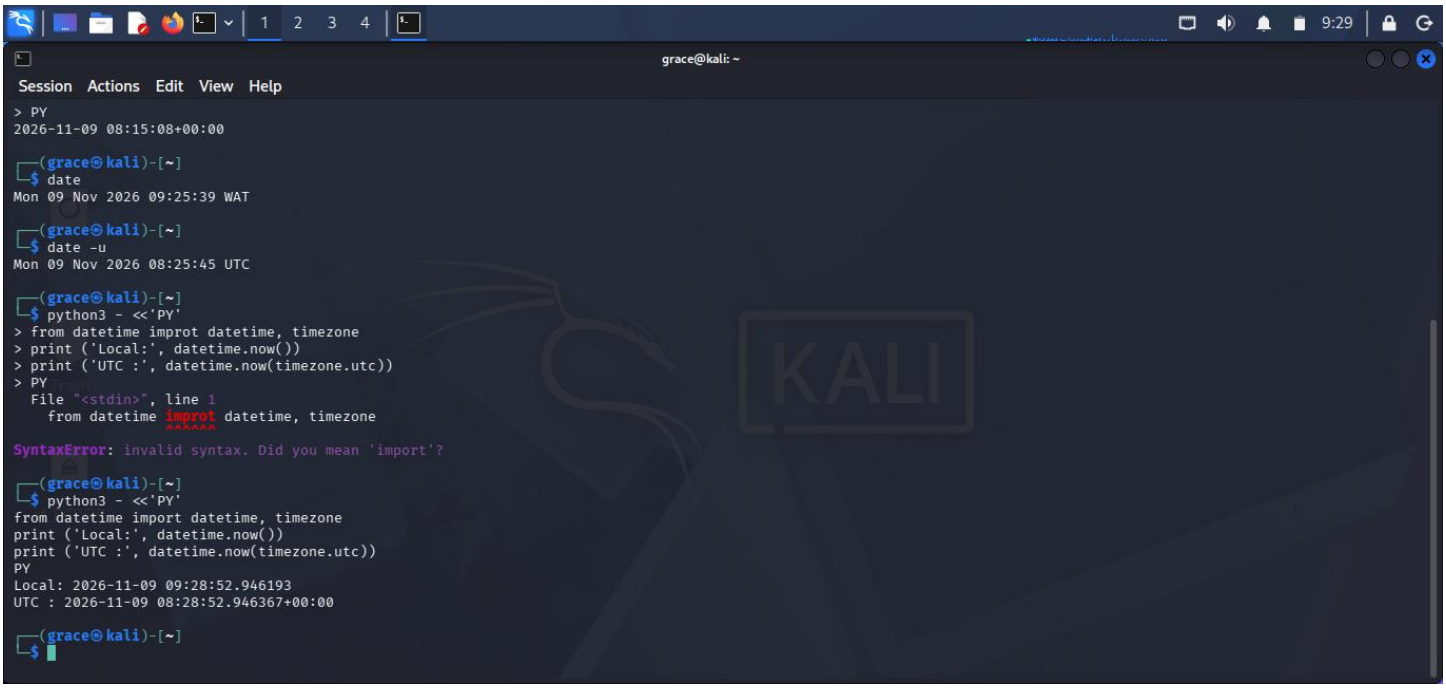


Fig.10

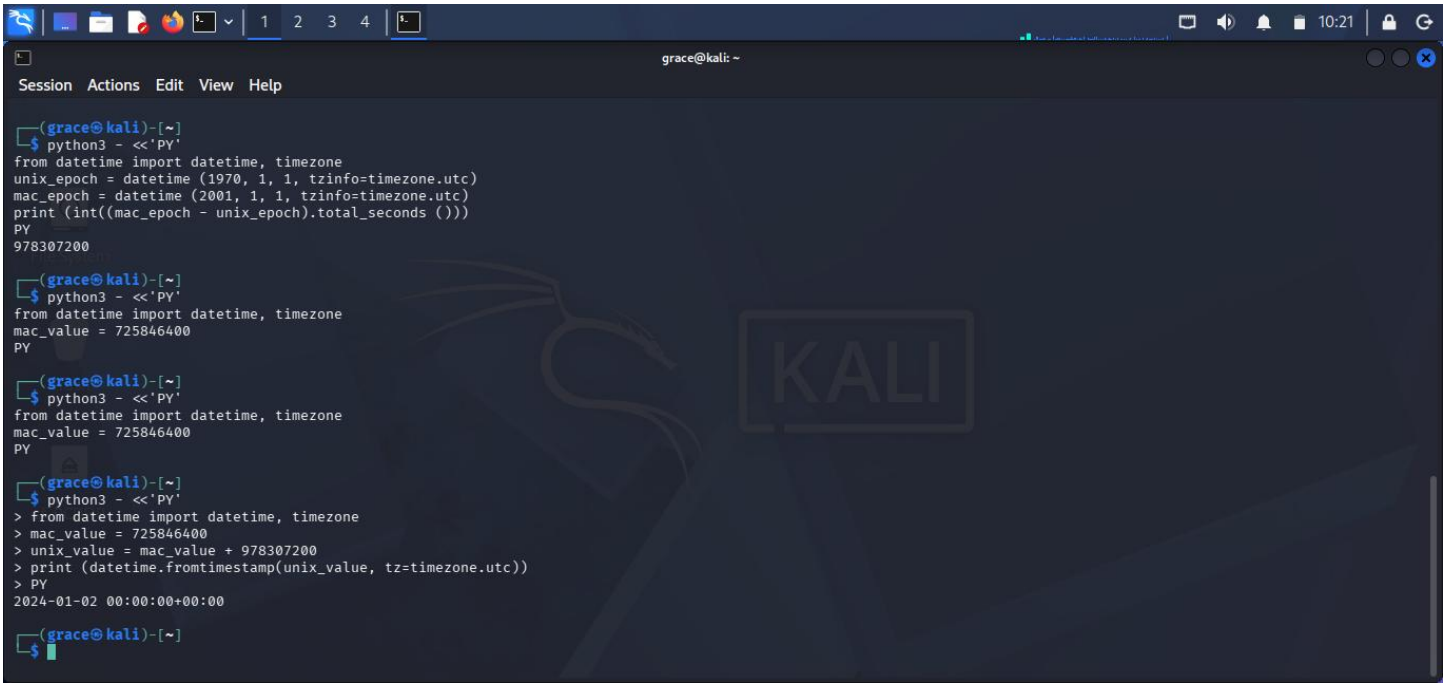


Fig.11

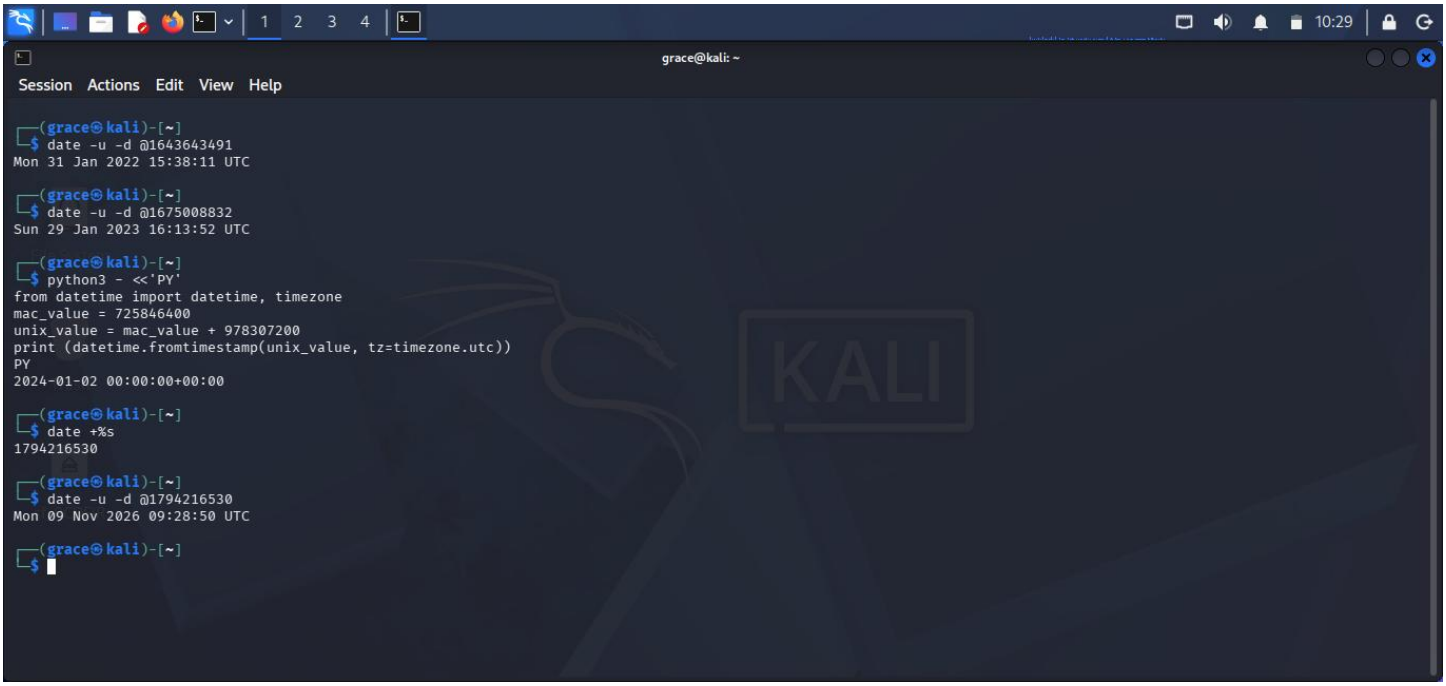


Fig.12

Task D - Hashing and the Newline Problem

Explain why the hash values differ: The hash values differ because the files do not contain exactly the same bytes. When echo is used without the -n option, it adds a newline character (0A in hexadecimal) to the end of the output. A file created with this extra newline therefore contains different data from a file created without it. Cryptographic hash functions are highly sensitive to changes in input, so even the addition or removal of a single byte produces a completely different hash value.

This is an important forensic point: **files that appear visually identical may still have different hashes because their underlying byte contents differ**.

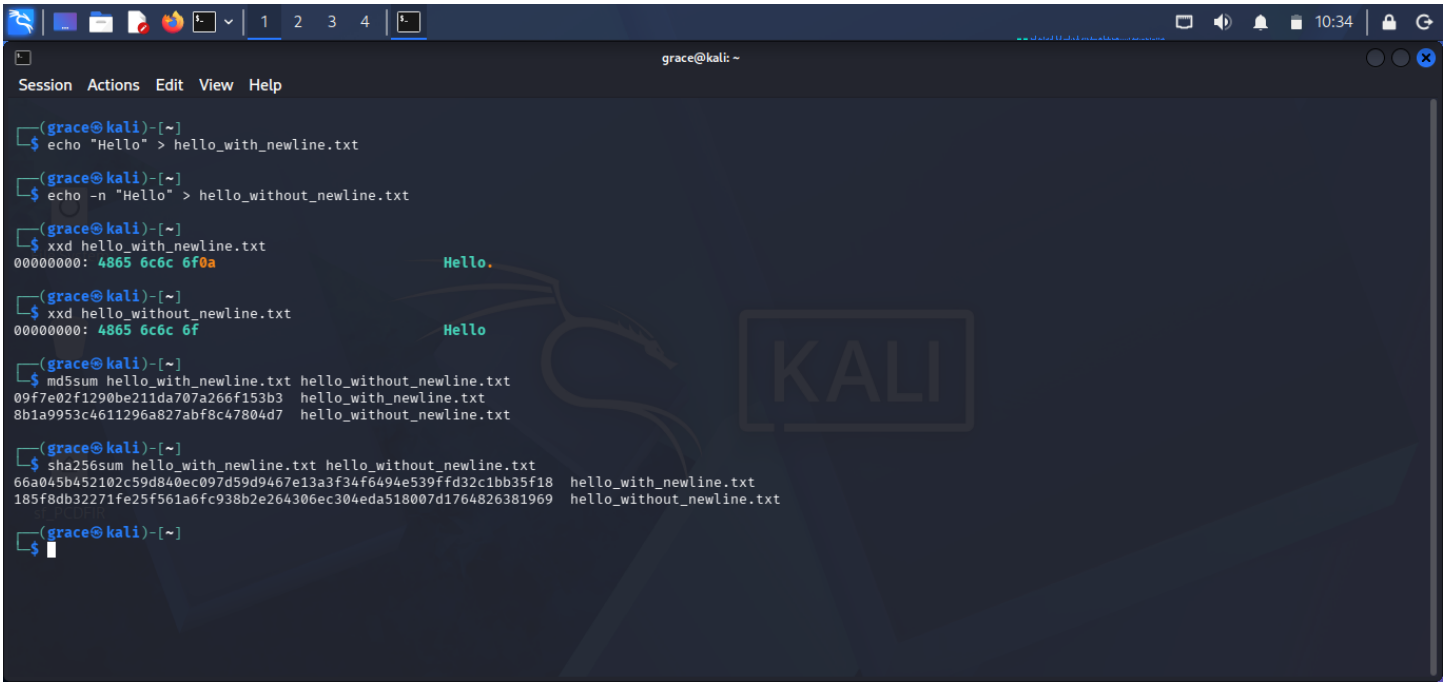

grace@kali: ~
Session Actions Edit View Help
(grace㉿kali)-[~]
$ echo "Hello" > hello_with_newline.txt
(grace㉿kali)-[~]
$ echo -n "Hello" > hello_without_newline.txt
(grace㉿kali)-[~]
$ xxd hello_with_newline.txt
00000000: 4865 6c6c 6f0a Hello.
(grace㉿kali)-[~]
$ xxd hello_without_newline.txt
00000000: 4865 6c6c 6f Hello
(grace㉿kali)-[~]
$ md5sum hello_with_newline.txt hello_without_newline.txt
09f7e02f1290be211da707a266f153b3 hello_with_newline.txt
8b1a9953c4611296a827abf8c47804d7 hello_without_newline.txt
(grace㉿kali)-[~]
$ sha256sum hello_with_newline.txt hello_without_newline.txt
66a045b452102c59d840ec097d59d9467e13a3f34f6494e539ffd32c1bb35f18 hello_with_newline.txt
185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969 hello_without_newline.txt
(grace㉿kali)-[~]
$

Fig.13

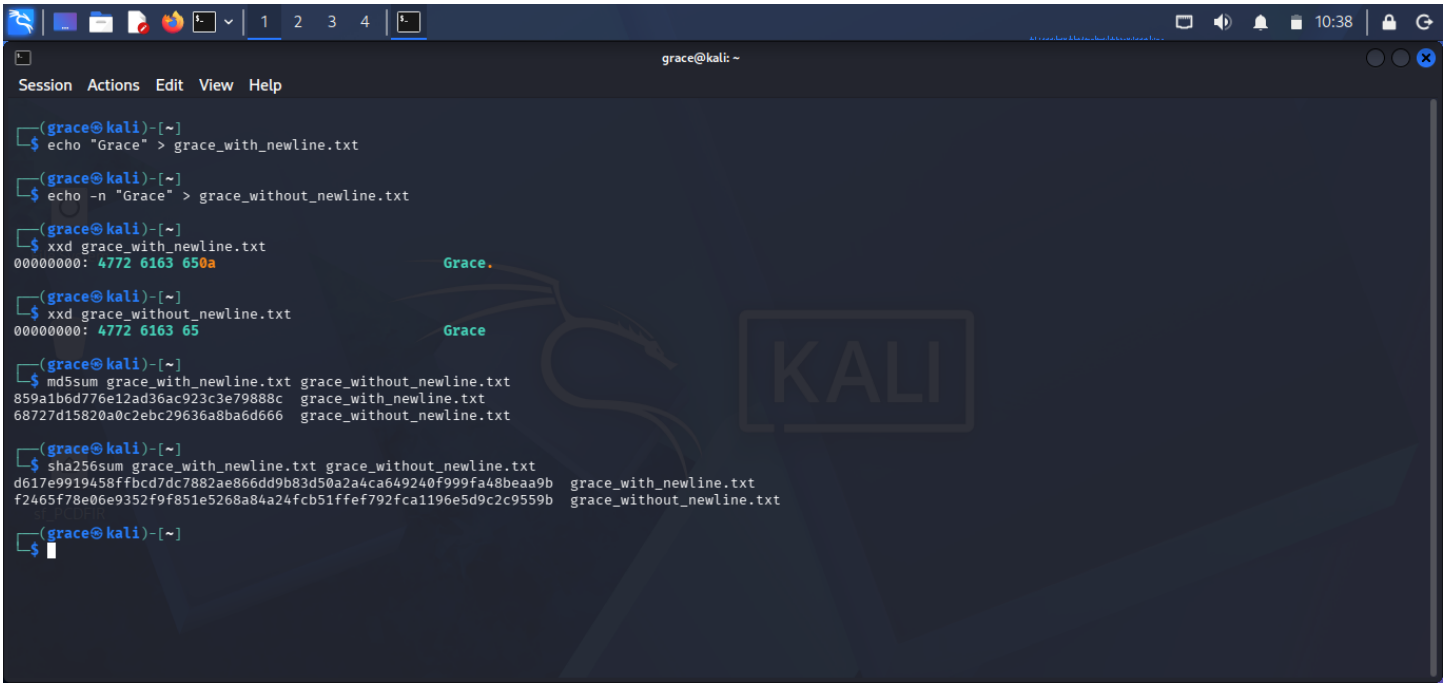

grace@kali: ~
Session Actions Edit View Help
(grace㉿kali)-[~]
$ echo "Grace" > grace_with_newline.txt
(grace㉿kali)-[~]
$ echo -n "Grace" > grace_without_newline.txt
(grace㉿kali)-[~]
$ xxd grace_with_newline.txt
00000000: 4772 6163 650a Grace.
(grace㉿kali)-[~]
$ xxd grace_without_newline.txt
00000000: 4772 6163 65 Grace
(grace㉿kali)-[~]
$ md5sum grace_with_newline.txt grace_without_newline.txt
859a1b6d776e12ad36ac923c3e79888c grace_with_newline.txt
68727d15820a0c2ebc29636a8ba6d666 grace_without_newline.txt
(grace㉿kali)-[~]
$ sha256sum grace_with_newline.txt grace_without_newline.txt
d617e9919458ffbcd7dc7882ae866dd9b83d50a2a4ca649240f999fa48beaa9b grace_with_newline.txt
f2465f78e06e9352f9f851e5268a84a24fcb51ffef792fca1196e5d9c2c9559b grace_without_newline.txt
(grace㉿kali)-[~]
$

Fig.14

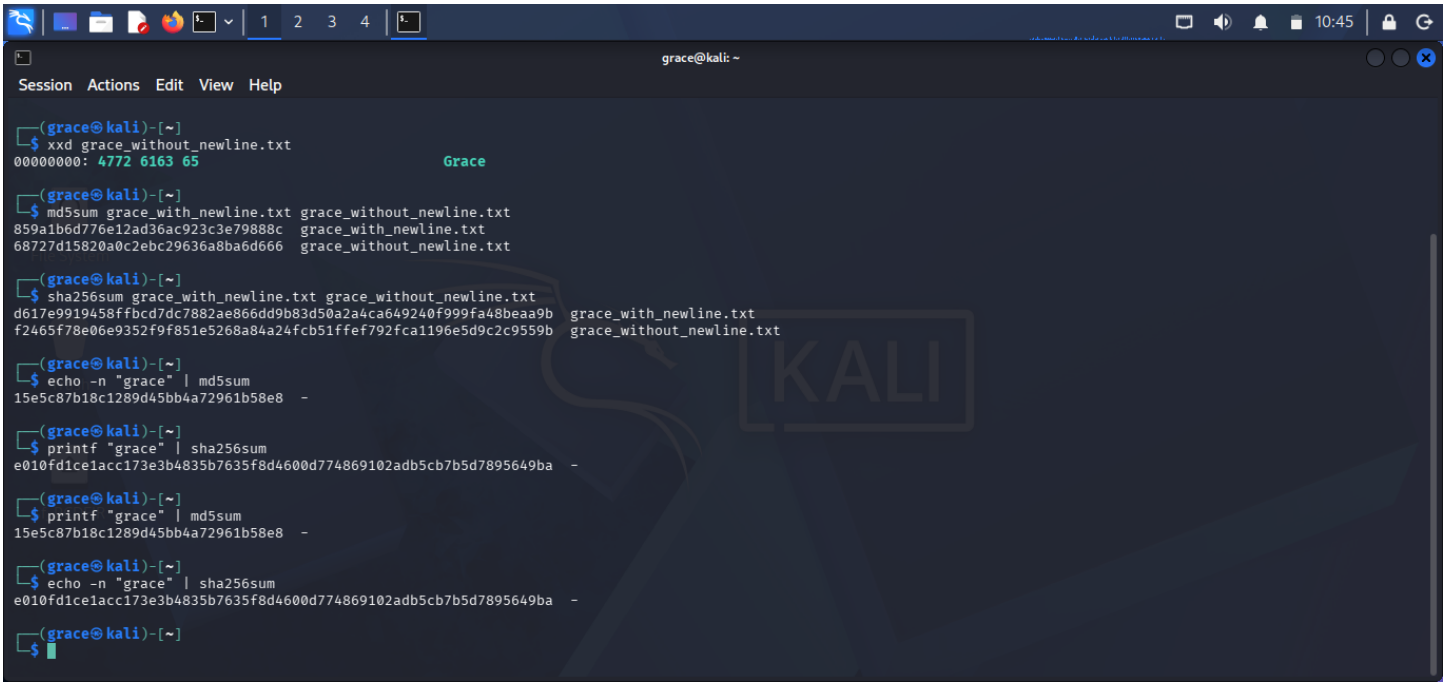


Fig.15

Task E - Endianness and Data Stored in Memory

explain how the same four bytes can produce different decimal values depending on byte order. Relate this to interpreting disk metadata or memory evidence: The same four bytes can produce different decimal values because little-endian and big-endian systems store or interpret the order of bytes differently. In little-endian format, the least significant byte is stored first, while in big-endian format, the most significant byte is stored first. Therefore, interpreting the same sequence of bytes using the wrong byte order can produce a completely different numerical value.

This is important in digital forensics because disk metadata, file-system structures, memory dumps, timestamps, and other binary evidence may depend on the correct byte order. An examiner should be able understand the format and architecture being analyzed; otherwise, a value such as a file size, address, timestamp, or metadata field could be interpreted incorrectly.

Raw Hex Bytes	Little-Endian Decimal	Big-Endian Decimal	Command Used	Explanation
01000000	1	16,777,216	Fig.16	
FF000000	255	4,278,190,080	Fig.16	
00000100	65,536	256	Fig,17	
78563412	305,419,896	2,918,915,346	Fig.17	
Forensic explanation requirement In your own words, explain how the same four bytes can produce different decimal values depending on byte order. Relate this to interpreting disk metadata or memory evidence.				

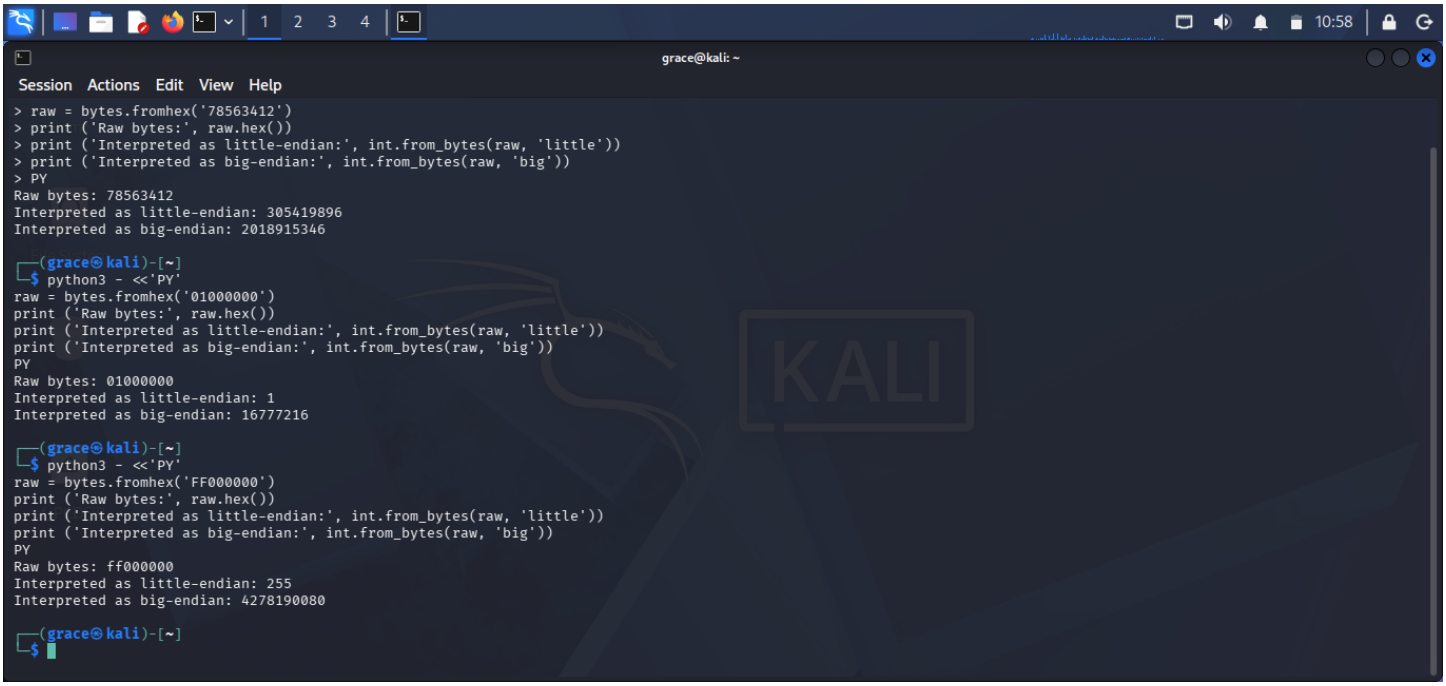


Fig.16

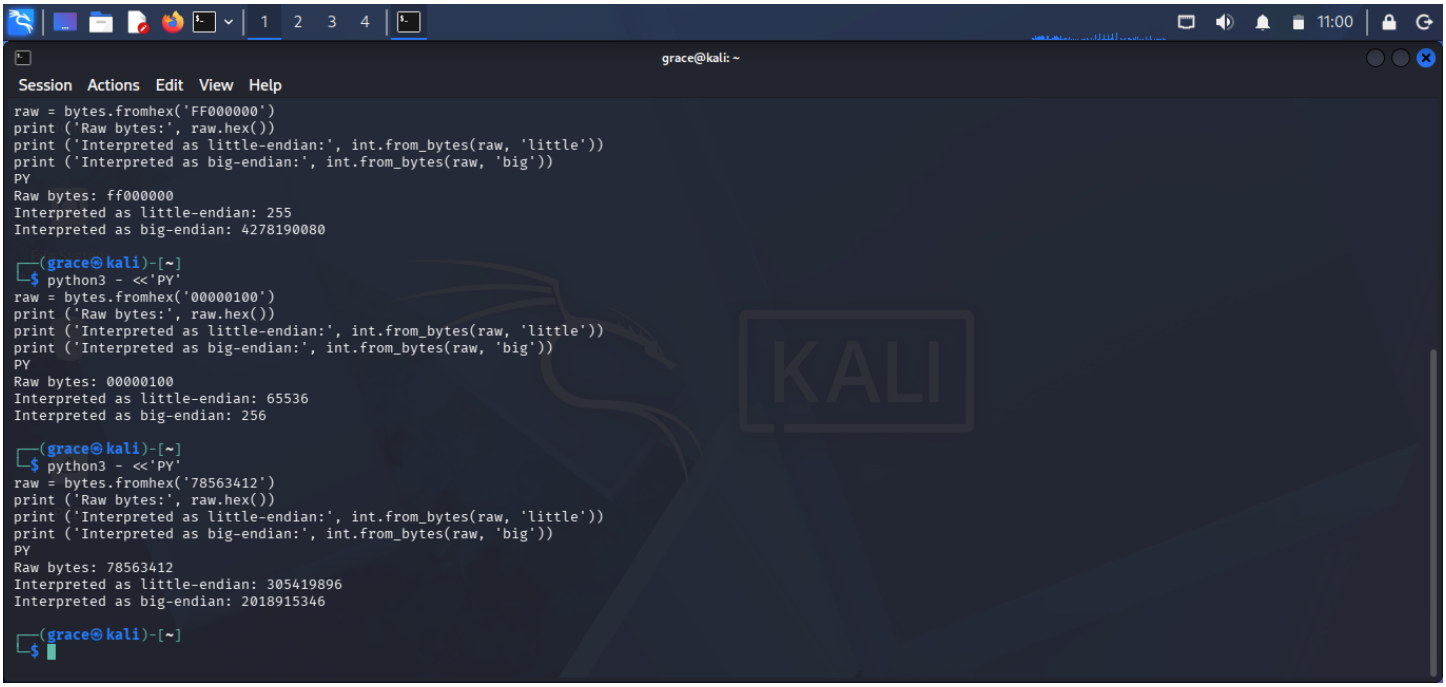


Fig.17

Task F - Mini Forensic Interpretation Case

- State what each evidence item represents after decoding – timeline analysis; text/content interpretation and byte order.
- For timestamp values, state the readable UTC time and the method used.
- For byte-order values, state both interpretations and why they differ – Little endian: 305,419,896 & Big endian: 201,891,5346. They differ because the same four bytes are interpreted in a different byte order.

- Explain which item would be useful for timeline analysis and which item would be useful for text/content interpretation – EVID-003 & 004 for timeline analysis, EVID-001 & 002 for text/content interpretation.

Evidence Item	Value	Student Interpretation	Command/Screenshot Ref.
EVID-001	48656c6c6f204943444641	Hello ICDFA	Fig.19
EVID-002	01001000 01101001	Hi	Fig.20
EVID-003	1675008832	Sun 29 Jan 2023 16:13:52 UTC	Fig.19
EVID-004	725846400 (Mac Absolute Time)	2024-01-02 00:00:00+00:00	Fig.19
EVID-005	78563412 as little-endian and big-endian	305419896; 2018915346	Fig.19

The evidence items demonstrate that raw digital data must be interpreted according to its format and context. EVID-001 and EVID-002 represent encoded text that can be decoded into readable content, making them useful for text and content interpretation. EVID-003 is a Unix timestamp which converts to 2023-01-29 16:13:52 UTC, while EVID-004 is a Mac Absolute Time value which converts to 2024-01-02 00:00:00 UTC. These timestamp values would be useful for reconstructing a forensic timeline of events. EVID-005 demonstrates endianness, as the same four bytes produce different decimal values depending on whether they are interpreted as little-endian or big-endian. This shows the importance of identifying the correct data format before drawing conclusions from digital evidence.

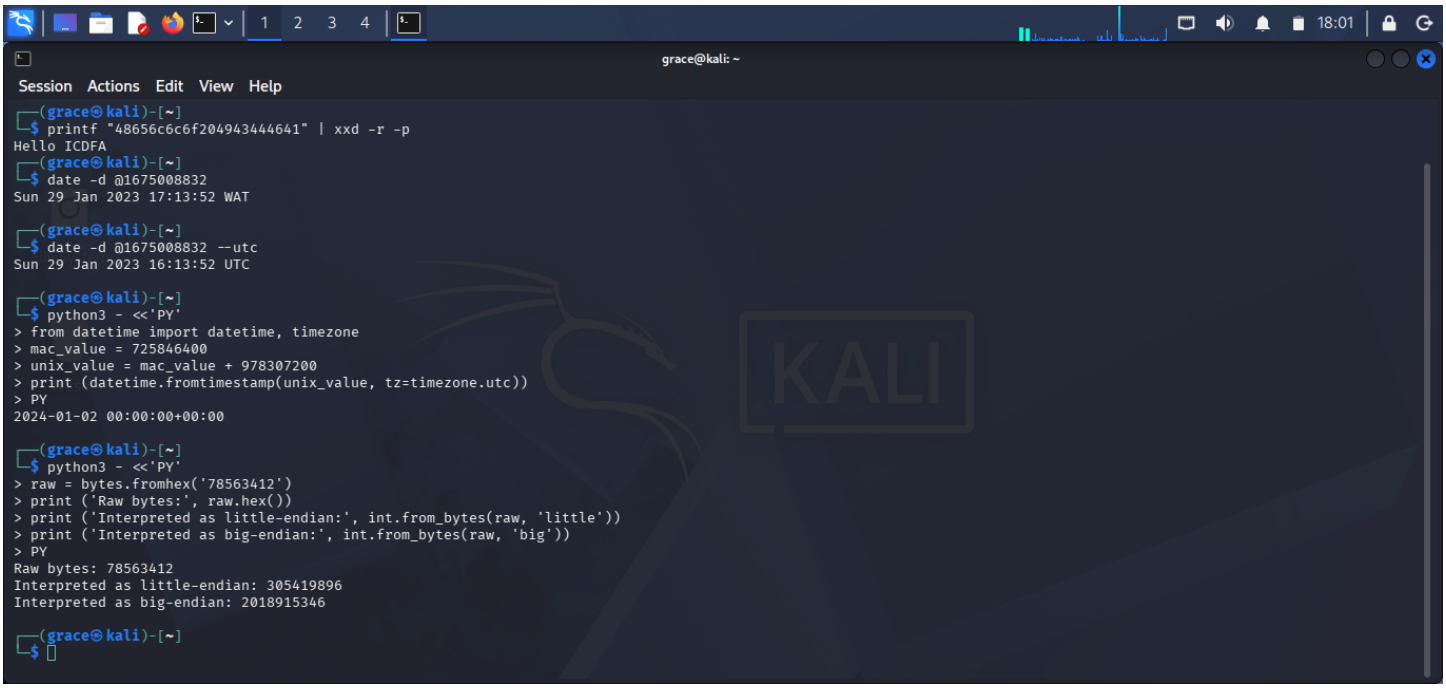


Fig.18

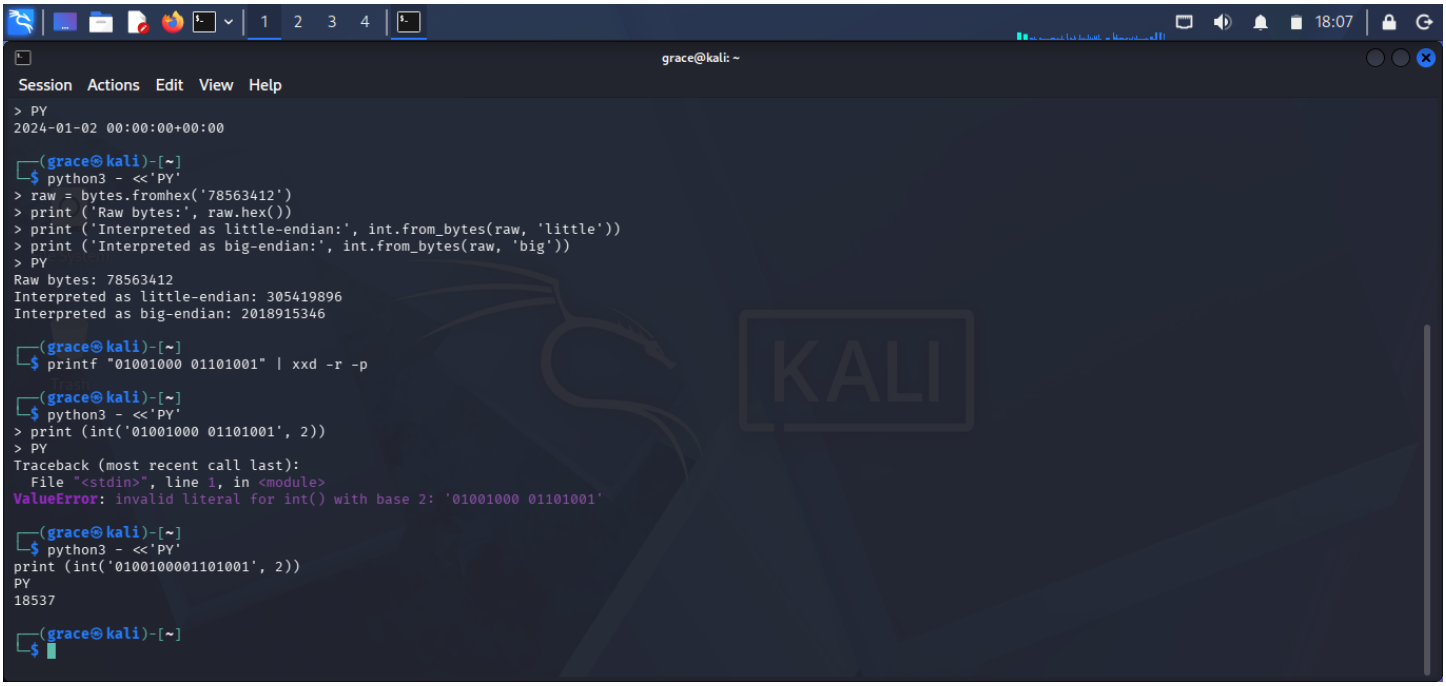


Fig.19

Reflection:

Number systems and timestamps are important in digital forensics because digital devices store and process information in binary form, while investigators must convert that information into meaningful evidence. Understanding binary, decimal and hexadecimal allow an examiner to correctly interpret raw data found in files, memory, disk structures and encoded evidence. Similarly, accurate timestamp conversion is essential for reconstructing the sequence of events during an investigation and establishing when specific activities occurred. Errors in interpreting number systems, byte order or time formats can lead to incorrect conclusions; therefore, careful validation and proper documentation of conversion methods are essential in forensic analysis.